

C++26 · P2300 · `std::execution` · A PRACTITIONER'S PERSPECTIVE

The Async Model is Here. Bring Your Own I/O (for now)

Infrastructure engineers should be very excited about where things are headed with C++26 and beyond. Here's what P2300 actually is, where the ecosystem stands today, and what's still genuinely missing.

~60 min · Code examples use `std::exec` · Familiarity with `std::future` and `std::thread` helpful

About me

- ▶ **Nick Milkovits** — Principal Software Engineer
- ▶ **Currently:** autonomous systems infrastructure for the past 6 years, focusing on the I/O layer
- ▶ **Previously:** almost a decade in high-frequency trading, working on front office core infrastructure where I/O performance is paramount
- ▶ **Tonight:** what P2300 means if you already own an async I/O layer

I'm coming at this from a **user and implementer perspective** — not as a proposal author or compiler vendor. What works, what doesn't, and what's still missing.

LinkedIn

linkedin.com/in/nicholasmilkovits

GitHub

github.com/catfinated

Views are my own. While my work in autonomous systems drives my interest in async I/O, this talk represents personal research and not the work or views of my employer.

Why I'm giving this talk

- ▶ I followed the executor proposals for a bit but **tuned out** when they missed C++20, then C++23.
- ▶ **Leahy's `std::execution` in Asio Codebases talk (CppCon 2025)** pulled me back in. A lot of this talk is me catching up on what I missed.
- ▶ I came home excited, tried to share it — and got **mixed to negative** feedback from colleagues and friends in similar roles.

These are good engineers running real systems. So the objections are worth answering rather than dismissing — and answering them is half of what tonight is.

WHAT I HEARD BACK

"I don't see how this is better than what we already have."

Depends on your workload. Own one async domain and it may not be.

"The pipes are just syntactic sugar around callbacks — `then` is morally the callback you'd have passed to `async_read` anyway."

Half right — and the half that's wrong is the whole point.

"Cancellation only happens at program termination. Errors get handled locally or terminate the process. And finalization can go in destructors."

Possibly correct for simple synchronous programs, but not async ones.

"It's too hard to use."

Largely conceded — WG21's long-serving convener acknowledges this.

01

THE PROBLEM

What problem is this actually solving?

Async I/O has been pulling at this for twenty years. The standard library gave us primitives, never a model — so we all built our own, and none of them compose.

The problem, concretely

Team A uses **Boost.Asio** for IPC. Team B uses **threads and futures** for CPU and GPU work. Team C uses **threads and io_uring** for file I/O.

You are asked to build the thing that uses all three: pull data off the network, process it, write the results to storage. **Good luck.**

- ✗ **No two of them compose.** Every pipeline across a boundary is a hand-written bridge — and a new one each time.
- ✗ **None of them share a vocabulary.** Each interface is its own; no generic algorithm can span them.
- ✗ **Each invents error and cancellation propagation.** Exceptions here, `error_code` there, silent drops somewhere else.

These aren't design mistakes — they're the ceiling **any** single-team library hits. Composing across teams needs a vocabulary everyone shares, a standard is a great place to provide one.

The pitch you'd make to these three teams: keep your Asio, keep your thread pool, keep your io_uring. Just agree on one type for **async work that hasn't finished yet** — a lazy, self-describing value. Then one set of generic algorithms works across all three, pipelines compose without anyone writing a bridge, and errors and cancellation cross the boundaries on their own.

Why `std::future` isn't the model — and what is

WHAT WE ALREADY HAVE

- ✗ **No continuations.** `.then()` was proposed in 2013 (N3721), shipped in the Concurrency TS in 2016, and never merged into the IS — chaining means blocking on `.get()` or spawning another thread.
- ✗ **No cancellation, no readiness callback.** You poll in a loop or you block.
- ✗ **No scheduler control.** `std::async` gives you one bit — new thread or lazy — and under the default policy even that bit is implementation-defined. You can never say **which** context.
- ✗ **Blocking fan-in, one error channel.** Join N futures serially. The only built-in failure path is `set_exception`, so an `error_code` I/O layer either throws `system_error` per `errno`, or smuggles results through `expected` and leaves the error channel unused.
- ✗ **No ownership.** A launched future is nobody's child. Drop it and the work runs on unwatched — there is no way to ask "is everything I started finished?"

WHAT A REAL MODEL NEEDS

- ✓ **Composable continuations.** Chain, fan-out, fan-in — one syntax regardless of what's underneath.
- ✓ **First-class cancellation.** Stopped is its own completion channel, not an exception in disguise.
- ✓ **Scheduler-agnostic.** Thread pool, GPU, io_uring ring, interrupt handler — the computation shouldn't care.
- ✓ **Uniform completion.** Value / error / stopped, propagated automatically through the chain.
- ✓ **Structured concurrency.** Every async operation has an owner and none outlives it — RAII's discipline, applied to work in flight rather than memory.

Each ✗ maps to the ✓ beside it. One more requirement we'll meet in the model section: **zero overhead on the default path** — allocation opts in, never sneaks in.

What about coroutines?

C++20 gave us the coroutine **mechanism** — `co_await` and `co_return` — but almost no library to use it with. The problems below aren't with coroutines themselves; they're with the missing infrastructure around them.

- ✗ **No standard scheduler — so no portability.** `cppcoro`, `Asio`, `folly`, `libunifex` each reinvented where coroutines run. A coroutine written against `Asio`'s executor doesn't compose with one written against your `io_uring` scheduler — so a library author cannot write one that works for both of you.
- ✗ **No structured cancellation.** Cancelling a coroutine chain requires library support — there's nothing in the language to propagate stop signals.
- ✗ **Fan-out is awkward.** `co_await` is sequential by nature; concurrent fan-out needs library primitives the standard doesn't have.

And one that isn't missing infrastructure: the frame. Elision is permitted, never required — and only fires when the coroutine is inlined and immediately awaited, not when the task is stored, passed or type-erased. A promise `operator new` controls **where**, never **whether**. A sender's operation state is in the caller's frame by construction: a guarantee, not a hope.

The good news: they interoperate

C++26 integrates them **both ways**: a coroutine `task` is a sender (P3552); any sender is awaitable via `as_awaitable` (P2300).

```
// Name the sender, then await it.
exec::task<int> my_coroutine() {
    auto doubled = stdexec::just(42)
        | stdexec::then([](int x) { return x * 2; });

    co_return co_await std::move(doubled);
}

// A coroutine task is itself a sender
auto work = stdexec::when_all(my_coroutine(), // sender ✓
                             other_work());  // sender ✓
```

Gotcha: `co_await` binds tighter than `|`. `co_await s | then(f)` awaits `s` and pipes the **result** into `then`. Parenthesize, or name it first.

02

THE MODEL

Three abstractions. One vocabulary.

Scheduler · Sender · Receiver — and why, as a user, you mostly only deal with one of them.

The model — three abstractions, three completions

No code yet. These are the only concepts the rest of the talk rests on.

THE THREE ABSTRACTIONS

 **Scheduler** — a handle to an execution context. **Where does work run?** Your `io_uring` becomes one.

 **Sender** — a lazy description of async work. You spend ~95% of your time here.

 **Receiver** — a generalized callback. **You almost never write one — but you'll call into one the moment you wrap your own I/O.**

THE THREE WAYS WORK COMPLETES — EVERY OPERATION ENDS ON EXACTLY ONE

`set_value(args...)`

Success. Zero **or more** values — a single completion can carry several independent results, which `std::future` cannot express at all.

`set_error(err)`

Failure. `exception_ptr`, `error_code`, or any type. Uniform propagation over the same channel, whatever the type.

`set_stopped()`

Cancellation. Structured and explicit — not an exception, not an error code.

And the property everything else follows from: senders are lazy values. Building a pipeline runs nothing — each step returns a new sender. Work starts only when a consumer connects and starts it. So a pipeline can be stored, returned from a function, passed to an algorithm, or handed to a scheduler its author never heard of.

The vocabulary to learn first

Thirty algorithms in C++26. These seven get you through almost everything — and three are just "read one of the completion channels."

FACTORIES — MAKE A SENDER OUT OF SOMETHING THAT ISN'T ONE

`just(vals...)` Completes immediately with the given values. The starting point when there's no scheduler involved. `just_error` / `just_stopped` complete on the other two channels.

`schedule(sched)` Completes on an execution context — the transition point. Everything piped after it runs there.

ADAPTORS — TAKE ONE OR MORE SENDERS, RETURN A NEW SENDER

`then(sndr, fn)` Reads the **value** channel. `fn` returns a value, which becomes the new value downstream.

`upon_error(sndr, fn)` Reads the **error** channel. Turns an error back into a value — recovery.

`upon_stopped(sndr, fn)` Reads the **stopped** channel. Turns cancellation into a value.

`let_value(sndr, fn)` Like `then`, but `fn` returns a **sender** — chaining a step that is itself async. `let_error` / `let_stopped` complete the grid.

CONSUMERS — CONNECT AND START IT. NOTHING RUNS UNTIL ONE OF THESE

`sync_wait(sndr)` Blocks until it completes. Returns `optional<tuple<values...>>` — empty on stopped, errors rethrow. Lives in `std::this_thread`.

Full cheatsheet — every algorithm, with signatures and gotchas:

github.com/catfinated/talks → [bring-your-own-io/sender-algorithms-cheatsheet.md](https://github.com/catfinated/talks/blob/main/bring-your-own-io/sender-algorithms-cheatsheet.md)

Longer treatment: **P4014**, "The Sender Sub-Language For Beginners" — all thirty algorithms with plain-C++ equivalents. [wg21.link/P4014](https://ericniebler.com/2021/04/14/p4014/)

Putting it together

Every algorithm here is one you just met.

```
// Nothing runs yet – this is just a description.
auto work = stdexec::just(path)
  | stdexec::let_value([](std::string& p) {
    // fn returns a SENDER, so the next step
    // can be async – and on another context
    return stdexec::schedule(io_sched)
      | stdexec::then([&p] { return read(p); });
  })
  | stdexec::then([](std::string s) { return parse(s); });

// Work STARTS here. This thread blocks until done.
auto [value] = stdexec::sync_wait(std::move(work)).value();
```

Two factories, two adaptors, one consumer. Each step returns a new sender and `sync_wait` is the only line that does anything. `let_value` keeps `p` alive for as long as the sender it returned — which is why the inner lambda can capture it.

```
// One factory and one adaptor per channel.
auto a = stdexec::just(std::string{"data"})
  | stdexec::then([](std::string s) {      // value
    return parse(s);
  });

auto b = stdexec::just_error(std::error_code{})
  | stdexec::upon_error([](auto err) {      // error
    log(err); return fallback();
  });

auto c = stdexec::just_stopped()
  | stdexec::upon_stopped([] {              // stopped
    cleanup(); return fallback();
  });
```

Recovery moves a completion back onto the value channel — which is why `b` and `c` return the same type their value path would have. Chain them instead of separating them and an error skips the rest of the value path to reach its handler: no `try / catch` at every step.

The two patterns that matter most

Fan-out with `when_all`

```
// Three operations run concurrently, on one pool.
exec::static_thread_pool pool{3};
auto sched = pool.get_scheduler();

auto work = stdexec::when_all(
    stdexec::schedule(sched)
    | stdexec::then([] { return load_config(); }), // Config
    stdexec::schedule(sched)
    | stdexec::then([] { return read_index(); }), // Index
    stdexec::schedule(sched)
    | stdexec::then([] { return fetch_stats(); })) // Stats
    | stdexec::then([](Config c, Index i, Stats st) {
        return build(c, i, st); // after all three
    });

auto [result] = stdexec::sync_wait(std::move(work)).value();
```

No blocking between steps. No thread-per-operation. One clean join point. **This shape is a graph** — remember it when we get to whether senders are worth it for your workload.

Scheduler hand-off with `continues_on`

```
// Two pools; a value handed from one to the other.
exec::static_thread_pool pool_a{2}, pool_b{2};

auto work = stdexec::schedule(pool_a.get_scheduler())
    | stdexec::then([] {
        return produce(); // on pool_a
    })
    | stdexec::continues_on(pool_b.get_scheduler())
    | stdexec::then([](auto v) {
        return consume(v); // on pool_b
    });

auto [r] = stdexec::sync_wait(std::move(work)).value();
```

Now replace `pool_b` with your `io_uring` scheduler. Nothing above or below that line changes — and errors and cancellation cross the boundary on their own.

Structured concurrency — fire-and-forget, still owned

Every **async operation** has an owner, and no operation outlives it. The discipline scoped variables and RAII gave us for memory, applied to work in flight.

`async_scope` is where you meet it.

```
// stdexec today
exec::async_scope scope;

scope.spawn(
    stdexec::starts_on(sched, do_work())
);
// caller continues immediately...

stdexec::sync_wait(scope.on_empty());
// ...but all spawned work is done here ✓
```

```
// C++26 — not just a rename
namespace ex = std::execution;
ex::counting_scope scope;

ex::spawn(ex::starts_on(sched, do_work()),
    scope.get_token());    // ← token

std::this_thread::sync_wait(scope.join());
```

`spawn` takes a **scope token**, not the scope — the algorithms are specified against a concept, so they work with any scope policy, including one you write. `counting_scope` is the policy the standard ships; `simple_counting_scope` is the same without stop support (P3149).

Forget to join and it calls `terminate()`. The destructor aborts unless the scope is joined, unused, or unused-and-closed. Correctness depends on remembering an idiom. **Hold that thought.**

So why not just join in the destructor? `join()` is async and a destructor can't be — it would have to **block**, and blocking there means waiting on work that may need the very context being torn down. The model keeps blocking at boundaries you can see: `sync_wait`, or a `join()` you wrote.

Why this concept gets so much airtime. It rests on the protocol's quietest rule: a receiver is completed **exactly once**, with one of value, error or stopped. That is what makes a counter reaching zero mean something — so a scope can promise that nothing it owns outlives it, and shutdown becomes provable rather than hopeful. Every other async model asks you to be careful; this one lets you be sure.

“Isn’t `then` just the callback?”

The second objection I promised to come back to. Same operation, both ways.

THE CALLBACK

```
struct ctx { std::vector<char> buf; };    // stop token? promise?

void on_read(int result, void* p) {
    auto* c = static_cast<ctx*>(p);
    if (result < 0) { // report to whom? }
    else process(c->buf, result); // on whose thread?
    delete c; // guaranteed heap-allocated?
}

// void async_read(int fd, std::span<char> buf,
//   void(*cb)(int result, void* ctx), void* ctx);
auto opstate = new ctx{};
async_read(fd, opstate->bufspan(), &on_read, opstate);
// nothing here to name, store, or compose
```

THE SENDER

```
auto work = async_read(iosched, fd, bufspan)
    | stdexec::then(process);

// ...and `work` is a value:
stdexec::sync_wait(std::move(work)); // run here
scope.spawn(std::move(work));       // or there
co_await std::move(work);            // inside a task
stdexec::when_all(std::move(work),   // or fan out
    other_work());
```

The consumer decides how it runs — not the algorithm. Same pipeline, four different executions, none of which `async_read` had to know about.

Where does the stop token go? There's one parameter left and it's `void*`. Same question for "which context does `process` run on?" and "who owns `buf` until completion? Change just about any detail and it's a different context+callback"

A callback is a function you give away. You hand it to `async_read` at the call site and it's gone. A sender is a value you keep.

The concession, and it's real: for one step, no errors, no cancellation, no fan-out — they **are** equivalent, and the callback version is less code. The value shows up at the second step, and at every seam between two libraries.

”Just do it in a destructor”

And the third — take it at face value and write the program it implies.

```
~BufferWriter() {  
    // flush what's still pending before we go  
    for (auto& b : pending_) {  
        ::write(fd_, b.data(), b.size());  
    }  
}
```

- ✗ **It doesn't know why it's running, so it can't know what to do.** Success, error, cancelled — same code, no parameter. `uncaught_exceptions()` is the only signal and cancellation isn't an exception, so a cancelled operation looks exactly like a successful one. After a failed or cancelled write `pending_` may hold a partial record: writing it corrupts the file, dropping it loses data.
- ✗ **And it can't tell anyone.** Destructors can't throw, so a failed flush is silent.
- ✗ **Blocking, serial, on the shutdown path.** No timeout — and routing it through your async layer means blocking on a context being torn down. Objects are destroyed one at a time in reverse order of construction, so ten writers with 50 ms of I/O each is **500 ms of serial teardown** that `when_all` would have finished in 50.

And the language forbids the fix: neither a constructor nor a destructor may be a coroutine — `[class.ctor.general]`, `[class.dtor]`. Not an oversight you can work around.

That is the thought I asked you to hold. `counting_scope` aborts because its cleanup is **async** and a destructor cannot be. A standard type whose correctness depends on an idiom — the C-style problem RAII was invented to kill.

So **async RAII is being built in the sender domain**. Shoop's `P2849`, now Leahy's `P3955` (Jan 2026) — implemented on `std::exec`, with an `io_uring` example that closes sockets via `IORING_OP_CLOSE`. On Linux even **close** is async.

03

THE ECOSYSTEM

Where things actually stand

The model shipped; the I/O didn't. What exists today, what it actually does, and the honest picture on standard library support.

The road to P2300

- 2005–07 Async I/O starts it all** — Boost.Asio appears (2005); N2175 (2007) **Networking Library Proposal for TR2**. Its need for a standardized executor model kicks off much of what follows. The Networking TS Draft grows from here.
-
- 2011 C++11** — `std::thread`, `std::future`, `std::async`. Useful primitives. No composable model.
-
- 2012–21 The executor wars** — years of competing proposals; P0443 (2016) **A Unified Executors Proposal for C++**, P1055 (2018) **A Modest Executor Proposal**, P1194 (2018) **The Compromise Executors Proposal**. N4771 (2018) **Working Draft, C++ Extensions for Networking** remains waiting for an executor model that never lands.
-
- 2018–19 Libunifex** developed at Facebook, open-sourced 2019. Eric Niebler and colleagues build a production sender/receiver system — the field-experience foundation for P2300.
-
- 2019–20 io_uring** lands in kernel 5.1. **C++20 coroutines** land as language features with no standard library support.
-
- 2021–23 P2300 RO** — Niebler, Leibach, Baker et al. submit the sender/receiver proposal; NVIDIA funds stdexec as the reference implementation. Direction settles on senders, the Networking TS is not pursued. P2762 (2023) **Sender/Receiver Interface For Networking**.
-
- Jun 2024 P2300R10 adopted** at WG21 St. Louis plenary — but narrowly: roughly a third of the room voted against, on grounds of complexity and teachability.
-
- Mar 2026 C++26 technical work completed** March 28 in London (Croydon). DIS ballot / ISO publication pending.

All along, underneath: every shop builds its own async framework from whatever primitives exist — or on Boost. That has been continuous since before C++11, it is still going on, and it is precisely why none of our systems compose.

The standard library situation

CHECKED 13 SEP 2026

C++26 is finalized. No shipping standard library exposes `std::execution`. Each claim below names the artifact it comes from — check them yourself, they move.

GCC / libstdc++

The `<execution>` header exists but only wires up C++17 parallel policies via Intel TBB. **No P2300 code in the source tree**; no public timeline. (GCC 16 shipped reflection and contracts first.)

source: libstdc++ tree + C++26 status page

Clang / libc++

The C++26 status page lists P2300R10 with a **blank status and no version number** — work hasn't started, or isn't tracked. No public timeline.

source: libc++ Cxx26 status page

MSVC / Microsoft STL

A tracking issue exists ([microsoft/STL #4768](#)) listing P2300R10 and its follow-ons with planned feature-test macros. The issue carries a note — **dated 2024** — that C++26 PRs won't be accepted until C++23 library work is finished.

source: microsoft/STL #4768 — re-check, this is the one most likely to have moved

`#include <execution>` today gets you C++17 parallel policies only — not P2300. Likely arrival: a few years, based on precedent — no vendor has published a timeline. **If you know otherwise, please say so!**

And C++26 execution is bigger than P2300: the standard also adopted `counting_scope` / `spawn` (P3149), a standard `parallel_scheduler` system context (P2079), a coroutine task type `std::execution::task` (P3552), and a policy-taking `bulk` (P3481). None of that has shipped in any stdlib either — but "the standard has no scheduler or task type" is no longer true on paper.

NVIDIA stdexec

[GITHUB.COM/NVIDIA/STDEXEC](https://github.com/NVIDIA/stdexec)

The reference implementation, now well beyond P2300 core. Header-only; requires C++20 or later.

In `stdexec/` — P2300 core:

- All core concepts, CPOs, and algorithms
- `run_loop` scheduler (single-thread event loop)
- Coroutine `as_awaitable` integration

In `exec/` — extensions beyond P2300:

- `static_thread_pool` — actual multi-thread scheduler
- `async_scope` — structured fire-and-forget with lifetime tracking (standardized as `counting_scope` in C++26)
- `when_any`, `repeat` / `repeat_until` / `repeat_n`, and `finally` — the last was proposed for C++26 in P3284 and `cut`; only `write_env` and `unstoppable` were adopted.
- `sequence_senders` — the real stream abstraction: `set_next`, `iterate`, `transform_each`, `ignore_all_values`. Very experimental, and the one thing a server needs that C++26 doesn't have.
- Type erasure: `any_sender_of`, `any_scheduler`
- `io_uring scheduler` — real scheduler: schedule, timers, stop propagation. No I/O **operations**.
- `asio/` — Leahy's asio integration library

In `nvexec/` — full CUDA / multi-GPU scheduler layer

Use this for experimenting with and evaluating sender-based compute pipelines today. The most complete implementation by far.

⚠ Experimental. API changes between releases. Not ABI-stable. NVIDIA explicitly say: not fit for any purpose whatsoever.

compiler-explorer.com has stdexec pre-installed as a library. Full runnable examples at github.com/NVIDIA/stdexec/tree/main/examples.

The Beman Project

[GITHUB.COM/BEMANPROJECT/EXECUTION](https://github.com/bemanproject/execution)

Volunteer-run, founded C++Now 2024. Goal: clean, spec-faithful implementations suitable for eventual upstreaming into libstdc++ / libc++.

What it implements:

- Essentially the **entire C++26 execution surface** — vocabulary and all algorithms, tracked clause-by-clause
- `just`, `then`, `let_value`, `starts_on`, `continues_on`, `when_all`, `bulk`, `sync_wait`
- Full scope suite (`counting_scope`, `spawn`, `spawn_future`) and coroutine `task` + `task_scheduler`
- `parallel_scheduler` (P2079) — the standard system context
- Requires C++23 (not C++20 like stdexec)

What it doesn't have:

- Nothing beyond the spec: no `when_any`, sequence senders, type erasure, or GPU
- No I/O — but `bemanproject/net` (P2762) is being built alongside

Watch this to track what will eventually land in the standard library. Spec-fidelity is the explicit primary goal.

Self-described: "under development and not yet ready for production use." Their clause-by-clause status doc shows implementation and tests essentially complete; review and docs still catching up.

`bemanproject/net` implements P2762 — sender-based networking, where standard async networking will come from. Early: only a `poll(2)` backend so far; `io_uring` / `epoll` / `kqueue` planned. C++29 target.

And it won't look like sockets. SG4 (Tokyo, Feb 2024) recommended **against** a low-level interface; the public API is to be based on **IETF TAPS** — describe what you need from a connection, the implementation picks the transport. Lower-level access stays underneath, not in front.

P2300 deliberately excludes I/O — here's what that means

Intentional, not an oversight. P2300 was scoped to the execution framework only; concrete I/O is a separate layer on top, and the authors have always said networking and file I/O are C++29 territory.

What `stdexec`'s `io_uring` integration **actually** is:

- A genuine scheduler — `schedule`, timers (`async_wait`), stop propagation
- Not a toy: the wiring compiles, runs, and cancels correctly
- But the repo example only demonstrates cancellation flow
- **No I/O operations:** no `async_read`, `async_write`, `async_accept`, `async_connect`

And a parallel effort that isn't sender-based at all: the C++ Alliance's **Network Endeavor** (P4100) — a paper series proposing coroutine-native I/O, with shipping implementations in Cappy and Corosio. Its authors call the two approaches complementary, not competing.

What actually exists for sender-based I/O:

- **Asio + Leahy's `stdexec` adaptation** — the only viable production path today. Real async file I/O (`stream_file`, `random_access_file`) and network I/O exposed as senders (CppCon 2025). Needs Asio; `io_uring` on Linux (`BOOST_ASIO_HAS_IO_URING`), IOCP on Windows.
- **`uring_exec`** — thin wrapper over every `io_uring_prep_*` op. Has read, write, accept, send, recv. Linux only. **Effectively archived since 2025** — build fixes only.
- **maikel/senders-io** — more designed file + network senders. Stale, old `stdexec` API. **This is where `stdexec` sends you:** [issue #1062](#) asked how to read files with the `io_uring` context in 2023; the maintainer's entire answer was a link to a header in this repo.
- **Beman net (P2762)** — the sender-based standards-track approach. Very early. C++29 target.
- **Intel bare-metal senders** — P2300 subset for embedded/interrupt-driven I/O. No OS required.

Already on Asio with `io_uring`? Leahy's adaptation is the most complete sender-based I/O available today. Everyone else is in DIY territory.

The platform picture is worse than it looks

Linux with io_uring BEST SUPPORTED

- io_uring is Linux-only (kernel 5.1+)
- Community projects exist (uring_exec, maikel/senders-io, and my own fiberexec) — research-grade, not production
- File I/O and network sockets both technically possible

Linux epoll / macOS kqueue DIY

- No sender-based scheduler wrapping epoll or kqueue exists
- You'd write your own — exposing completions as `set_value()`
- Niebler's blog "What are Senders Good For?" is the guide

Windows IOCP NOTHING

- No sender-based IOCP scheduler exists publicly
- Asio wraps IOCP — so Leahy's Asio adaptation is the path for now

Embedded / bare-metal PARTIAL

- Intel's bare-metal senders: a P2300 subset — interrupt-driven, no OS, no alloc
- Shows the model generalizes to metal; separate library, not stdexec-interoperable

If you already have your own I/O layer on your platforms of choice, then senders give you the model to plug into.



THE BRIDGE

Bring your own I/O.

The model is here and the I/O isn't. So what do you actually do about it? It depends.

Adoption is workload-dependent

COMPUTE-HEAVY ASYNC WORK

(parallel pipelines, GPU offload, CPU-bound task graphs)

- stdexec's story is **good here today**
- Thread pool + GPU schedulers are the most mature part
- `when_all`, `bulk`, `nvexec` all work well in practice
- Consider adopting stdexec now if you can tolerate API churn

HPC, scientific computing, GPU pipelines — this is the most compelling case for adoption today. The model was built for this use case first.

I/O-HEAVY ASYNC WORK

(network servers, file pipelines, embedded I/O, storage systems)

- The composition model is sound and ready
- The I/O primitives are **not production-ready**

Starting from scratch? Senders are not the go-to yet — you'd be building the I/O layer **and** the wrapper, and the standard gives you neither. Put senders above it if you need composability with the compute layers; inside it, use senders, coroutines, callbacks — whatever fits.

Already have an async I/O layer? Expose it as senders and you get the whole model — structured cancellation, uniform completion, scopes — plus composability with every other sender-shaped library. The `io_uring` / `epoll` / `socket` code underneath doesn't change.

Sharper test than compute vs I/O — is your async work a graph, or a chain? Fan-out, joins, work crossing a ring and a pool and maybe a GPU: that's a graph, and the algebra earns its complexity. Read a frame, parse it, write a response: that's a chain, and a coroutine expresses it with far less machinery. Most I/O-heavy systems are some of both — the seam is where you decide.

If you already own an async I/O layer, wrap it behind a sender-shaped seam. That is the recommendation, it pays off today, and it survives whichever way the standard lands. **How much you build on top of it** is what depends on your workload. **So — how do you wrap it?**

The bridge: wrapping your existing async I/O as senders

If you have an `io_uring` scheduler, `epoll` loop, or any callback-based system — expose it as a scheduler today and gain composability without rewriting anything.

```
// Your existing C-style async API:
// void async_read(int fd, char* buf, size_t len,
//   void(*cb)(int result, void* ctx), void* ctx);

// Wrap it as a sender:
struct read_sender {
    using sender_concept = stdexec::sender_tag;
    using completion_signatures = stdexec::completion_signatures<
        stdexec::set_value_t(char*, std::size_t),
        stdexec::set_error_t(std::error_code),
        stdexec::set_stopped_t()>; // you cancel — declare it

    int fd; char* buf; std::size_t len;

    auto connect(stdexec::receiver auto rcvr) && {
        return read_op<decltype(rcvr)>{fd, buf, len, std::move(rcvr)};
    }
};

// Now your I/O flows into any sender pipeline:
auto work = read_sender{fd, buf, 4096}
    | stdexec::then(parse) | stdexec::then(process);
```

This is the practical adoption path **right now** for I/O engineers: keep your existing async I/O layer, expose it as a sender, gain composability at the computation level today. **This part is boilerplate — the next slide is where the heavy lifting happens.**

Niebler's blog "What are Senders Good For, Anyway?" (Feb 2024) walks this pattern in detail — wrapping a C-style async API to produce senders. Essential reading.

When standard I/O senders eventually arrive (P2762, C++29+), you swap the wrapper for the standard primitive. The pipeline logic above doesn't change.

The operation state — where the real work lives

```
// using namespace stdexec; — elided below for width
template <stdexec::receiver R>
struct read_op {
    using operation_state_concept = operation_state_tag;
    read_op(read_op&&) = delete;           // pinned: built in place

    void start() & noexcept {
        // 1. hook the receiver's stop token into YOUR layer
        stop_.emplace(get_stop_token(get_env(rcvr_)), on_stop{this});
        // 2. submit; `this` is the completion context
        ::async_read(fd_, buf_, len_, &on_complete, this);
    }

    static void on_complete(int result, void* ctx) noexcept {
        auto& self = *static_cast<read_op*>(ctx);
        self.stop_.reset();                // unregister BEFORE completing

        if (result ≥ 0)
            set_value(std::move(self.rcvr_), self.buf_, std::size_t(result));
        else if (result == -ECANCELED)
            set_stopped(std::move(self.rcvr_));    // not an error!
        else
            set_error(std::move(self.rcvr_),
                      std::error_code(-result, std::system_category()));
    }

    int fd_; char* buf_; std::size_t len_; R rcvr_;
    using stop_cb_t = stop_token_of_t<env_of_t<R>>::template callback_type<on_stop>;
    std::optional<stop_cb_t> stop_;
};
```

Pinned, caller-owned storage. `connect()` constructs the op state in place and it never moves. This is where "no allocation on the default path" actually comes from — it isn't a slogan, it's this `= delete`.

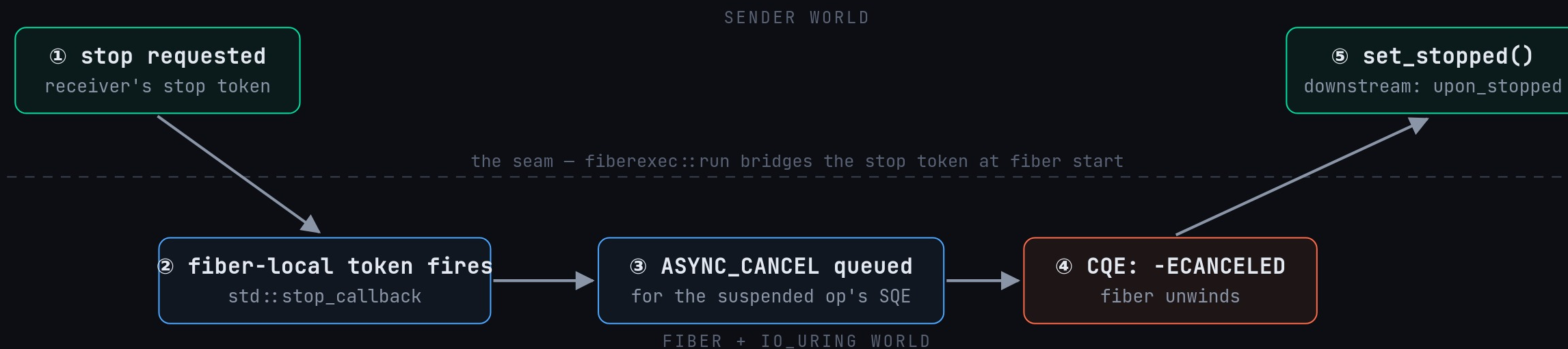
Stop callback lifetime. The receiver carries an **environment** — a bag of queries the caller populates, which is how the stop token, scheduler and allocator reach you without appearing in any signature. Registered in `start()`, torn down before the completion fires. Get that ordering wrong and you have a use-after-free on the cancellation path — the bug that doesn't show up until production.

Three-way completion is the whole game. `-ECANCELED` maps to `set_stopped`, not `set_error`. Collapse those two and every `upon_stopped` downstream is dead code.

Before you wrap: two things to watch out for

Watch out: older stdexec examples and early P2300 talks use `tag_invoke` for customization — `friend auto tag_invoke(connect_t, ...)` etc. That mechanism was replaced in P2300R10, the final adopted revision. The C++26 standard uses plain member functions: `connect()`, `start()`, `set_value()`, and `using sender_concept = sender_tag` — renamed from `sender_t` at Croydon (P4154R0), along with `receiver_tag`, `scheduler_tag` and `operation_state_tag`. stdexec keeps the old spellings as aliases.

Cancellation: wire the receiver's stop token into your layer's own cancellation so `set_stopped()` is real, not decorative — it's where hand-rolled wrappers are most likely to cut corners. Here is the flow from `fiberexec`, a personal research runtime where I did this integration end to end — fibers inside, senders outside:



The unwind is fiber-internal — it's how a suspended fiber abandons its stack without checking a return code at every await point. It never crosses the seam; the sender world only ever sees `set_stopped()`.

Building bridges is the expected path

And the last objection — the one I conceded at the start. Here is who else concedes it.

std::execution is "currently harder to adopt than most C++ features."

— Herb Sutter, March 2026 Croydon trip report. Convener of WG21 2002–2025, now convener emeritus; technical fellow at Citadel Securities.

He attributes that to thin documentation and missing small convenience libraries, and tells new adopters to budget learning time with someone who already knows it — and to expect to **write their own adapter layers** bridging `std::execution` to their existing async code. He also notes his own company is already running it in production.

That last instruction is the whole of tonight's talk — from the person who convened the committee for 23 years. **Not a workaround he tolerates; the route he tells you to take.**

What he doesn't address: for non-GPU, non-thread-pool workloads the adapter you're expected to write is **the entire I/O layer**. That's the gap to plan around — not a reason to ignore P2300, a reason to be clear-eyed about the timeline.

Summary

- ▶ **Why:** every pre-P2300 async codebase is correct but isolated — the standard gave us primitives, never a model.
- ▶ **What:** senders are lazy, composable descriptions of work; the scheduler is swappable — your `io_uring` ring can be one.
- ▶ **The gap:** C++26 is done, but no `std::lib` ships it (a few years out) — and P2300 deliberately excludes I/O.
- ▶ **The bridge:** wrap your existing async I/O layer as senders today; swap in standard I/O when P2762 lands.
- ▶ **Verdict:** learn now, plan carefully. Compute-heavy: adopt. I/O-heavy: adapt, don't rewrite.

My advice: don't delay — start learning now. The model itself isn't the hard part; adoption is. But the payoff is worth it: **composable asynchronous libraries lead to composable asynchronous systems.**

END OF TALK

Questions?

To get started on Godbolt: compiler-explorer.com → add library **stdexec** → explore the examples at github.com/NVIDIA/stdexec/tree/main/examples

If you're using raw sockets, `io_uring`, `epoll`, or any callback-based async system and want to think through wrapping it as senders — let's talk.

Experiment

godbolt.org + **stdexec** · examples on GitHub

Read

Niebler blog · Teodorescu ACCU Dec 2024

Watch

bemanproject.net · P2762 · libc++ status

Wrap

Your async layer → senders → composability

References — reading & talks

Slides will be shared — no need to transcribe.

Essential reading

- **"Senders/Receivers: An Introduction"** — Teodorescu, ACCU Overload Dec 2024. Best written intro available.
- **"What are Senders Good For, Anyway?"** — Niebler, blog Feb 2024. The guide to wrapping C-style async APIs.
- **cppreference.com/w/cpp/execution** — final C++26 wording. The reference.
- **P4014 "The Sender Sub-Language For Beginners"** — Falco & Gill. All thirty C++26 sender algorithms, each with an example and its plain-C++ equivalent. CCO public domain. Read §2.4 for the theory mapping.

Talks

- "Think Parallel" — Bryce Lebach, ACCU 2024. Best accessible motivation talk.
- "Working with Asynchrony Generically" 1&2 — Niebler, CppCon 2021. Canonical, API dated.
- "std::execution in Asio Codebases" — Leahy, CppCon 2025. The migration path for existing async code.
- "Extending std::execution: Higher-Order Senders" — Leahy, C++ on Sea 2025. The composition frontier.
- "Embedded Async Abstraction Without an OS" — Deane, C++Now 2024. Bare-metal angle.
- **Leahy, C++Now 2026 — "Towards Async Everything", 2 parts on YouTube.** Part 1 is io_uring; part 2 is P3955. Closest thing to a companion to tonight's talk.
- **CppCon 2026:** Leahy, "Implementing Async RAI" — the P3955 design, implemented on stdexec.
- **New in 2026:** "The Async Pattern Graveyard" — Leahy, ACCU on Sea 2026: how 20 years of Asio lessons shaped senders. Plus two further Leahy sender sessions and Ažman on C++26 concurrency at C++Now 2026 — recordings through summer 2026.

References — code & what to watch

Repositories

- github.com/NVIDIA/stdexec — use this for experimentation and evaluation now
- github.com/bemanproject/execution — watch for stdlib direction
- github.com/bemanproject/net — P2762 sender networking, very early
- github.com/maikel/senders-io — stdexec+io_uring networking and file i/o (outdated)
- github.com/Caturra000/uring_exec — stdexec+io_uring network lib (archived)
- intel.github.io/cpp-baremetal-senders-and-receivers — embedded use case
- github.com/catfinated/fiberexec — my research runtime: io_uring + fibers + senders
- catfinated.com/talks — these slides, a **sender algorithm cheatsheet**, and a **buildable demo project** with every example from tonight. Some material adapted from P4014R2 (Falco & Gill), CCO 1.0.

Will standard async I/O arrive?

- **P2762** — sender networking proposal, C++29 target. Track its revisions.
- **P3482** — Rodgers/Kühl, "Design for C++ networking based on IETF TAPS". What the API will **look** like — describe the connection you need, not the socket.
- **P4003** — the Network Endeavor's execution model. The coroutine-first bet.
- Beman net commit activity — leading indicator of stdlib direction
- WG21 mailings for any file I/O sender proposals in the C++29 cycle

Where the model goes next — and whether it ships

- **P3955** — Leahy, async RAI for senders: async construction and destruction. **R1 has wording**; SG1 saw RO at Brno and asked to see it again with wording.
- **P4320** — Leahy, `std::execution::sequence`: senders run one after another (**not** streams). R1, August 2026.
- **P4007** — Falco/Gill, open issues in `std::execution::task`. Informational; §3 lists what shipping forecloses.
- **cppreference C++26 support** — the "Execution control library" row. Watch for a version number to appear in any column.

”Could I write a network server with this?”

First, a terminology trap. **Single-shot** vs **multi-shot** is about how many times a sender can be `connect`ed — not how many values it delivers. `just(42)` is multi-shot because it's copyable; a coroutine task is single-shot; `exec::split` converts one to the other (cut from C++26 by P3682; a `stdexec` extension now). **Every sender completes exactly once.** Multi-shot is not "recv until the peer closes."

Streams need a different abstraction: sequence senders. One emits many items via `set_next` before a final completion. `stdexec` has them in `exec/sequence/` — `iterate`, `transform_each`, `ignore_all_values`, `merge_each`.

Not in C++26. No paper. Nothing in Beman. I'm told they are actively being worked on.

Don't confuse these with `std::execution::sequence` (Leahy's P4320, based on `exec::sequence`) — that runs senders one after another. Sequential composition, not streaming. The naming is unhelpful.

The lifetime story is ready. A server's fan-out is unbounded and runtime-determined — N connections, each outliving the accept iteration that made it. That is what `counting_scope` exists for: spawn each connection through `scope.get_token()`, keep accepting, shut down with `request_stop()` then `join()`.

THREE WAYS TO WRITE THE LOOP TODAY

- **Coroutine per connection.** An `exec::task` with an ordinary `while` loop over `co_await async_recv(...)`. Readable by anyone; one frame allocation per connection, which is noise next to a socket. **What I'd reach for.**
- `repeat_until` — the loop in pure sender form. (Was `repeat_effect_until`; that spelling is now deprecated.) Pitfall: it needs the sender to be either copyable or lvalue-connectable, which type-erased senders generally aren't. Erase a sender **factory** instead.
- **Sequence senders** — `accept_seq | transform_each(handle) | ignore_all_values`. Experimental; open bugs include type-erased sequences not propagating stop tokens.

"Static pipelines only" is half wrong. The **types** are static and monomorphized; the **graph** is not. `let_value` picks the next sender at runtime, `spawn` gives runtime child count, `any_sender_of` erases shape, coroutines give arbitrary control flow. The dynamic parts cost a frame or an erasure allocation — the zero-allocation property covers the statically-shaped parts only.

“Can I build the graph at runtime, from a config file?”

Yes — and the shape of the answer is the same as every other dynamic-shape question. It works; you pay for it at each dynamic edge.

```
// One erased vocabulary type for every node kind
using node = exec::any_sender_of<
    set_value_t(Frame),
    set_error_t(std::error_code),
    set_stopped_t()>;

// kind → factory, populated at startup
std::map<std::string,
    std::function<node(Params const&)>> registry;

node build(YamlNode const& cfg) {
    node s = registry.at(cfg.kind)(cfg.params);
    for (auto const& child : cfg.children)
        s = std::move(s) | then(adapt(child)); // re-erased
    return s;
}
```

What each dynamic edge costs. An allocation — the op state's size is no longer known — and no inlining across the seam. Every node kind must share **one** completion signature set.

The compile-time contract stops helping. Signatures were checked while the graph lived in the type system; a YAML-driven graph moves those errors to runtime.

Runtime-N fan-out needs a different tool. `when_all` is variadic — compile-time arity — and `std::exec` has no `when_all` over a vector. Spawn each branch into a `counting_scope` and join, or collect `spawn_future` results. Also: erased senders aren't copyable, so `repeat_until` rejects them — erase a **factory** instead.

The alternative: codegen. Emit the pipeline as C++ from the config at build time and everything stays monomorphized.

The fair comparison isn't against static senders — it's against what you'd pay anyway. A config-driven graph in a callback or virtual-dispatch design costs an indirect call and an allocation per node too. Senders aren't worse here; they just aren't better, because you've opted out of the thing that makes them fast.

What “in the caller’s frame” actually means

`connect()` builds one object. Each adaptor's operation state **contains** its child's — so the whole pipeline is a single nested value with a size known at compile time.

```
stdexec::sync_wait(
  async_read(sched, fd, buf, len)
| stdexec::then([](char* p, std::size_t n) {
    return std::string{"hello "}
      + std::string{p, n};
  }));
```

Two senders, **one object, zero allocations**. Its size is the sum of the boxes on the right, fixed at compile time.

This is why the op state is immovable: the outer state holds the inner one **by value**. Move it and every stashed pointer — including the one the kernel is holding — dangles.

CALLER'S STACK FRAME

sync_wait op state

receiver → result storage · run_loop

then op state

sync_wait's receiver · the lambda, by value

read_op – yours

rcvr_ – then's receiver, pointing back out

fd_ · buf_ · len_ · stop_ callback

the SQE's user_data points at this box

no child – the recursion stops here

connect()
nests inward

set_value(p,n)
back out

then vs let_value

The whole rule: `then`'s callable returns a **value**; `let_value`'s returns a **sender**. Reach for `let_value` whenever the next step is itself asynchronous.

THEN — SYNCHRONOUS TRANSFORM

```
just(x) | then([])(int x) {
    return x * 2;          // a value ✓
};
// downstream sees: set_value(int)
```

Runs on whatever context the previous step completed on. No new async work is introduced.

LET_VALUE — CHAIN IN ASYNC WORK

```
just(path) | let_value([&](string& p) {
    return async_read(sched, p);
    // ↑ a sender ✓
})
| then([])(Buffer buf) {
    return parse(buf);
});
```

The returned sender is connected and started; the pipeline's completion is **its** completion. This is how you nest one async operation inside another.

The common mistake: using `then` with a callable that returns a sender. It compiles — and gives you a value completion carrying an unstarted sender that nobody ever connects. The work silently never runs.

Structured concurrency — safety by construction

THE OLD WAY — DANGER

```
// Thread may outlive `data`
std::thread([&data] {
    process(data); // dangling ref! ✨
}).detach();
// data destroyed here
```

Detached threads are a well-known footgun. No way to know when — or whether — they complete.

THE SENDER WAY — SAFE

```
// Work is nested — cannot outlive scope
stdexec::sync_wait(
    stdexec::starts_on(sched,
        stdexec::just(data)
        | stdexec::then(process)
    )
);
// Guaranteed: process() is done here ✓
```

Child work completes before the parent scope exits — and P2300 **removed** the escape hatches (`start_detached` , `ensure_started`) before shipping, so the unstructured version is no longer a one-liner.

Ecosystem at a glance

Reference for Q&A — not presented live.

Feature	stdexec (NVIDIA)	Beman execution	libstdc++ / libc++
C++26 execution vocabulary & algorithms	✓	✓	✗ not yet
sync_wait, coroutine task	✓	✓	✗
Thread pool / system scheduler	✓ static_thread_pool, parallel_scheduler	✓ parallel_scheduler	✗
Structured scopes (counting_scope / spawn)	✓	✓	✗
when_any, finally	✓	✗	✗
Type erasure (any_sender_of)	✓	✗	✗
io_uring scheduler	✓ scheduler + timers + stop; no I/O ops ⚠	✗	✗
Async file / network I/O ops	✗ community projects only	net: early, poll(2) only	✗
GPU / CUDA (nvexec)	✓	✗	✗
Production-ready	⚠ experimental	✗ self-stated	✗ not shipped